🎙️ Fichas de Exposición Rápida (10 Minutos) — QuickFood ERP

Guía Sintética para Grabación en Vivo (100% Código, Cero Relleno)

COTECNOVA 2026 • Software de Gestión Empresarial (Seminario RAD Laravel)

Integrantes: Miguel Ángel Ramírez Gutiérrez | Andrés Felipe Mejía Suaza

> 💡 **Regla de Oro para el 5,0:** No lean párrafos largos. Hablen fluido con estas frases clave de 2 líneas. Mencionen las clases, archivos y comandos mientras muestran la pantalla. Tienen un celular con cronómetro al lado para no pasarse del minuto 10:00.

⏱️ RESUMEN CRONOLÓGICO RÁPIDO

Minuto	Tema	Quién Habla	Ventana que debe estar en pantalla
0:00 – 1:00	Presentación y Arquitectura Backend	**Miguel** (0:00-0:30) **Andrés** (0:30-1:00)	VS Code (árbol de carpetas) y Dashboard
1:00 – 2:30	Modelo de Datos y Reglas en Código	**Miguel** (1:00-1:45) **Andrés** (1:45-2:30)	Diagrama MER y migraciones en VS Code
2:30 – 6:30	Demostración en Vivo (Código + BD)	**Miguel** (2:30-3:30 / 4:30-5:30) **Andrés** (3:30-4:30 / 5:30-6:30)	Terminal MySQL + VS Code + Navegador
6:30 – 8:30	Seguridad Spatie y Middlewares	**Miguel** (6:30-7:30) **Andrés** (7:30-8:30)	`RolePermissionSeeder.php`, `SaleController` y Error 403
8:30 – 9:30	Inmersión en `SaleService.php`	**Miguel** (8:30-9:30)	VS Code (`DB::transaction` y `lockForUpdate`)
9:30 – 10:00	Pruebas PHPUnit y Cierre	**Andrés** (9:30-10:00)	Terminal con `php artisan test` (22 tests en verde)

📄 FICHAS DE APOYO MINUTO A MINUTO

FICHA 1 (Minuto 0:00 – 1:00): Presentación y Arquitectura

- 💻 **En pantalla:** VS Code mostrando carpetas `app/Http/Controllers`, `app/Services`, `app/Models`.
- 🎙️ **MIGUEL (0:00 – 0:30):**

> *"Buenas noches docente y compañeros. Somos **Miguel Ángel Ramírez** y **Andrés Felipe Mejía**, y sustentamos el Segundo Corte de Software de Gestión Empresarial en COTECNOVA.*
> *Presentamos la arquitectura de **QuickFood ERP**, diseñado para una Dark Kitchen con reparto a domicilio. Desarrollado en **Laravel 11** bajo arquitectura desacoplada: Controladores, Capa de Servicios transaccionales y Modelos Eloquent."*

- 🎙️ **ANDRÉS (0:30 – 1:00):**

> *"En una cocina oculta el problema crítico es la concurrencia: evitar sobreventas de platos agotados en horas pico.*
> *Lo resolvimos con transacciones atómicas `DB::transaction()`, bloqueo pesimista `lockForUpdate()` en MySQL, control de acceso basado en roles con Spatie y métricas en tiempo real en Eloquent."*

FICHA 2 (Minuto 1:00 – 2:30): Modelo de Datos y Reglas de Negocio

- 💻 **En pantalla:** Diagrama Entidad-Relación y migraciones `create_sales_table.php` en VS Code.
- 🎙️ **MIGUEL (1:00 – 1:45) — [Modelo y Relaciones]:**

> *"Extendimos el esquema relacional con migraciones en Laravel:*
> *1. Tabla `clients` con campos tributarios y bandera `active`.*
> *2. Cabecera en `sales` vinculada 1 a N con `sale_details` con claves foráneas estrictas: `constrained()->onDelete('restrict')`, impidiendo borrar clientes con facturas.*
> *3. Cada detalle guarda snapshots inmutables: cantidad, precio unitario y subtotal vinculados a `productos`.*
> *4. Y aplicamos el trait `SoftDeletes` (`deleted_at`) en ventas, productos y clientes para auditoría contable."*

- 🎙️ **ANDRÉS (1:45 – 2:30) — [3 Reglas Clave en Código]:**

> *"Implementamos 3 reglas de negocio en el backend:*
> *• **Regla 1 (Bloqueo Pesimista):** En `SaleService`, cada ítem se consulta con `Product::lockForUpdate()`, bloqueando la fila en InnoDB para evitar colisiones concurrentes.*
> *• **Regla 2 (Validación Atómica y Rollback):** Si `stock < cantidad` o el producto está inactivo, lanzamos `ValidationException` y la transacción ejecuta Rollback automático.*
> *• **Regla 3 (Devolución en Anulación):** En `SaleController@destroy`, se ejecuta `$product->increment('stock')` y borrado lógico con `$sale->delete()`."*

FICHA 3 (Minuto 2:30 – 6:30): Demostración Funcional en Vivo (Código y BD)

- 💻 **En pantalla:** Terminal MySQL interactiva + VS Code + Navegador.
- 🎙️ **MIGUEL (2:30 – 3:30) — [Paso 3.1: Stock en MySQL y Form Request]:**
- *Comando en MySQL:* `SELECT id, nombre, stock FROM productos WHERE id = 1;` *(Stock = 50).*
- *Mostrar en VS Code:* `app/Http/Requests/StoreSaleRequest.php`.

> *"Consultamos en MySQL: el producto 1 tiene actualmente **50 unidades**.*
> *Al enviar el POST a `/sales`, `StoreSaleRequest` valida que el cliente exista, esté activo y no tenga soft-delete mediante `Rule::exists('clients', 'id')->whereNull('deleted_at')->where('active', true)`, y valida que el array `items` contenga enteros positivos."*

- 🎙️ **ANDRÉS (3:30 – 4:30) — [Paso 3.2: Transacción, Descuento en MySQL e IVA]:**
- *Acción:* Registrar venta de 2 unidades en el navegador.
- *Comando en MySQL:* `SELECT id, nombre, stock FROM productos WHERE id = 1;` *(Stock = 48).*

> *"El controlador invoca `SaleService::register()`. El código calcula el IVA del 19% con `round($subtotal * 0.19, 2)`, genera el consecutivo `FV-00001` con `str_pad` y ejecuta `$product->decrement('stock', 2)`.*
> *Comprobamos en MySQL: el stock bajó en caliente de 50 a **48 unidades** de forma atómica."*

- 🎙️ **MIGUEL (4:30 – 5:30) — [Paso 3.3: Sobreventa y Rollback Atómico]:**
- *Acción:* Intentar vender 999 unidades en el navegador (muestra error).
- *Comando en MySQL:* `SELECT id, nombre, stock FROM productos WHERE id = 1;` *(Stock sigue en 48).*

> *"Intentamos facturar 999 unidades. En `SaleService`, la condición `if ($product->stock < $quantity)` se cumple y lanza `ValidationException`.*
> *Al propagarse en `DB::transaction()`, el motor ejecuta un **Rollback inmediato**. Verificamos en MySQL: el stock sigue exactamente en **48 unidades**, sin datos corruptos ni saldos negativos."*

- 🎙️ **ANDRÉS (5:30 – 6:30) — [Paso 3.4: Anulación con Increment, Factura PDF y KPIs]:**

- *Acción:* Mostrar factura `/sales/show`. Presionar "Anular Venta".
- *Comando en MySQL:* `SELECT id, nombre, stock FROM productos WHERE id = 1;` *(Stock vuelve a 50).*
- *Mostrar:* `/dashboard`.

> *"En `sales/show.blade.php` la factura incluye estilos `@media print` para exportar a PDF.*
> *Al anular la venta, `SaleController@destroy` ejecuta `$product->increment('stock', 2)` y borrado lógico con `$sale->delete()`.*
> *En MySQL vemos que el stock retornó a sus **50 unidades iniciales**.*
> *Y en el Dashboard, `DashboardController` calcula el inventario en tiempo real con `sum(DB::raw('price * stock'))` y el Top 5 con `groupBy('product_id')->orderByDesc('total_sold')`."*

FICHA 4 (Minuto 6:30 – 8:30): Seguridad, Roles Spatie y Middlewares

- 💻 **En pantalla:** `RolePermissionSeeder.php`, `SaleController.php` y pantalla de Error 403 en navegador.

- 🎙️ **MIGUEL (6:30 – 7:30) — [Configuración Spatie y HasMiddleware]:**

- *Mostrar en VS Code:* `database/seeders/RolePermissionSeeder.php` y `SaleController.php`.

> *"La seguridad se implementó con `spatie/laravel-permission`.*
> *En `RolePermissionSeeder.php` definimos roles: `admin`, `vendedor`, `almacenista` y asignamos permisos modulares con `$role->syncPermissions()`.*
> *En `SaleController` implementamos la interfaz nativa `HasMiddleware`, definiendo `new Middleware('permission:ver-ventas', only: ['index', 'show'])`. Ninguna petición no autorizada puede llegar al controlador."*

- 🎙️ **ANDRÉS (7:30 – 8:30) — [Prueba en Vivo de Error 403 y Papelera]:**

- *Acción en navegador:* Iniciar sesión como `almacenista@test.com` y forzar en la URL: `http://localhost:8090/sales`.
- *Mostrar pantalla:* **403 | USER DOES NOT HAVE THE RIGHT PERMISSIONS**.

> *"En las vistas Blade usamos directivas `@can('ver-ventas')` para ocultar opciones del menú.*
> *Para probar la seguridad en el servidor: iniciamos sesión como Almacenista (sin permiso de ventas) y forzamos la URL `/sales`.*
> *El middleware de Spatie intercepta la petición y dispara una excepción `UnauthorizedException`, retornando este error **HTTP 403 Forbidden**.*
> *Además, desde el rol Administrador gestionamos la papelera con `Product::onlyTrashed()` y restauramos con `$product->restore()`."*

FICHA 5 (Minuto 8:30 – 9:30): Inmersión en Código Transaccional (`SaleService`)

- 💻 **En pantalla:** VS Code en pantalla completa en `app/Services/SaleService.php` (método `register()`).

- 🎙️ **MIGUEL (8:30 – 9:30) — [Propiedades ACID y Row-Level Locking]:**

- *Señalar con el cursor:* `DB::transaction()` y `Product::lockForUpdate()`.

> *"Analizamos el núcleo del servicio en `SaleService.php`.*
> *El método `register()` ejecuta toda la lógica dentro de `DB::transaction()`, cumpliendo las propiedades **ACID**:*
> *• **Atomicidad:** Se inserta cabecera, detalles y descuento de stock, o se hace rollback total.*
> *• **Aislamiento:** La instrucción `Product::lockForUpdate()` impone un bloqueo pesimista en MySQL (`SELECT ... FOR UPDATE`), evitando condiciones de carrera si dos usuarios compran en simultáneo.*
> *• Y la función `nextInvoiceNumber()` consulta `Sale::withTrashed()->orderByDesc('id')` con `str_pad` para garantizar consecutivos fiscales unívocos sin saltos."*

FICHA 6 (Minuto 9:30 – 10:00): Pruebas Automatizadas y Plan Tercer Corte

- 💻 **En pantalla:** Terminal ejecutando: `php artisan test`.
- 🎙️ **ANDRÉS (9:30 – 10:00) — [22 Tests en Verde y Cierre]:**
- *Acción en terminal:* Correr `php artisan test`.

> *"Para certificar que toda la lógica es tolerante a fallos, construimos 22 pruebas automatizadas en PHPUnit.*
> *Como observamos en consola, `php artisan test` arroja un resultado del **100% de tests en verde**: validamos transacciones, rollbacks, claves foráneas, errores 403 y cálculo de KPIs.*
> *Para el Tercer Corte implementaremos: auditoría con Eloquent Observers, API RESTful con Laravel Sanctum para repartidores móviles, y colas de trabajo (Queues) para facturación electrónica.*
> *Muchas gracias por su atención."*

⚡ COMANDOS RÁPIDOS PARA TENER COPIADOS:

```
-- 1. En la consola MySQL (Sail):
SELECT id, nombre, stock FROM productos WHERE id = 1;
```

```
-- 2. En la terminal de Laravel:
php artisan test
```

Fichas diseñadas para una exposición fluida, profesional y cronometrada en 10:00 minutos exactos.
COTECNOVA 2026 — Miguel Ángel Ramírez Gutiérrez & Andrés Felipe Mejía Suaza.